S&P 500 Market Direction
An End-to-End MLOps Pipeline (Proof of Concept)

Miguel Venâncio António Varela Maria Cruz Carlos Amorim

NOVA Information Management School (NOVA IMS), 2025/2026 • github.com/vehnie/sp500-mlops-pipeline

1 Data Choice, Objectives & Success Metrics

Why this data. We use daily OHLCV history of the S&P 500 index (Yahoo Finance, ticker `^GSPC`, 2000–2026, 6,604 usable rows). The choice is deliberate for an *MLOps* project rather than a pure modelling one: the data is free, clean, reliably available, and updates every trading day, so it realistically exercises ingestion, scheduling, and drift monitoring. It is also a genuine time series, which forces correct, leak-free engineering (chronological splits, time-aware validation and monitoring), and it represents a *low-signal* problem, which is exactly where solid operational tooling earns its value: small pipeline mistakes can masquerade as "performance", so contracts and tracking are essential.

What we try to achieve. The task is binary classification: predict whether the next trading day closes higher than today, $y_t = \mathbb{1}[\text{Close}_{t+1} > \text{Close}_t]$. The real objective, however, is to deliver a **production-grade ML lifecycle**: reproducible orchestration (Kedro), experiment tracking and a model registry (MLflow), data contracts (Great Expectations), drift monitoring (Evidently), explainability (SHAP), a served REST API (FastAPI + Docker), and automated tests (pytest). The model itself is intentionally simple so the engineering around it is the deliverable.

Success metrics. We track success on two levels (Table 1). *Model metrics* are Accuracy, Precision, Recall, F1, and ROC-AUC, with **ROC-AUC as the primary discrimination metric** since it is threshold-independent and robust to the mild class imbalance. *System (MLOps) metrics* define what "done" means operationally and are, for this PoC, the more important bar.

Table 1: Success criteria and final status.

Level	Criterion	Status
Model	ROC-AUC reported honestly on a leak-free chronological test set	✓ 0.499
Model	Champion beats challenger on the selection metrics (Acc/Recall/F1)	✓
System	One-command reproducible run (`kedro run`) from raw to predictions	✓
System	Data contracts pass at raw and feature layers	✓ 19/19, 27/27
System	Model tracked, versioned, and registered with lineage (MLflow)	✓
System	Drift monitor produces an actionable report	✓
System	Champion served behind a REST API + container	✓
System	Automated test suite green	✓ 80 passed

2 System Architecture & Data Quality

The system is a directed acyclic graph of **Kedro** pipelines, each a set of nodes with declared inputs/outputs registered in a central data catalog, so every artifact is traceable to the node that produced it. A single `kedro run` executes the default end-to-end workflow:

`data_quality` → `data_cleaning` → `data_feat_engineering` → `data_split` → `model_train` → `model_train_challenger` → `model_predict` → `model_predict_challenger`

Three further pipelines, `data_expectations` (contracts), `model_explainability` (SHAP), and `data_drift` (Evidently), run on demand to generate the validation, interpretability, and monitoring artifacts. Table 2 maps each MLOps concern to its implementing tool. **Data quality** is

enforced with Great Expectations at two checkpoints: a *raw contract* (OHLCV schema/types, non-null prices, `High`$\geq$`Low`, positive volume, valid dates) and a *feature contract* (all eight features finite, `rsi_14`$\in [0, 100]$, target $\in \{0, 1\}$). Both pass fully in the final run (19/19 and 27/27), so a future data refresh that breaks an assumption fails loudly and early.

Table 2: MLOps concerns and the components implementing them.

Pipeline orchestration & catalog — **Kedro**	Drift monitoring — **Evidently**
Experiment tracking & registry — **MLflow**	Model serving (REST API) — **FastAPI + Uvicorn**
Data & feature contracts — **Great Expectations**	Containerisation — **Docker / Compose**
Explainability — **SHAP** (`LinearExplainer`)	Automated testing — **pytest** (80 tests)

3 Project Planning (Agile Sprints)

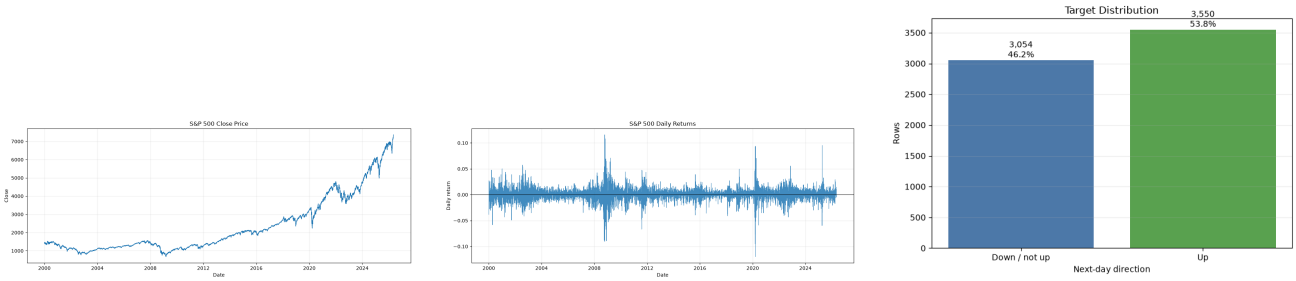
We organised the work as six one-week sprints with a Kanban board of pipeline-sized tasks. Each sprint produced a runnable increment: a new Kedro pipeline plus its tests and catalog entries were the "definition of done". This kept the system end-to-end runnable at every stage and let monitoring/serving be added without touching the core training path.

Table 3: Sprint plan. Each sprint delivers one or more registered Kedro pipelines.

Sprint	Goal	Key deliverables
S0	Setup & exploration	Repo, Kedro scaffold, `data_ingestion`, EDA notebooks
S1	Data foundation	`data_cleaning`, `data_feat_engineering`, `data_split` (chronological 70/15/15), target definition
S2	Data quality	`data_quality` + `data_expectations` (Great Expectations contracts)
S3	Modelling & tracking	`model_train` (LR champion), `model_train_challenger` (RF), MLflow tracking + registry
S4	Evaluation & explainability	`model_predict(_challenger)`, reporting plots, `model_explainability` (SHAP)
S5	Monitoring, serving & hardening	`data_drift` (Evidently), FastAPI + Docker, Streamlit, pytest suite, docs/report

4 Results: Exploration & Modelling

Data exploration. The index level is strongly trending and *non-stationary*, while daily returns are stationary around zero with clustered volatility (Figure 1). The target is mildly skewed toward up-days ($\approx 53\%$), consistent with long-run equity drift. From the price and volume we engineer eight leak-free features (all using information up to day t): `simple_return`, `log_return`, `sma_10`, `sma_20`, `sma_ratio_10`, `rsi_14`, `volatility_10`, `volume_change`. The set is kept small so every feature stays inspectable.



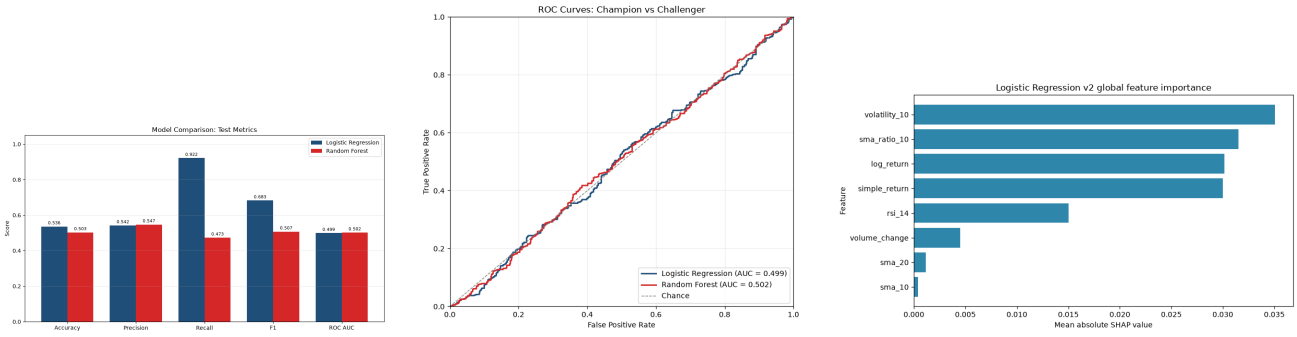
(a) Close price (non-stationary). **(b)** Daily returns (stationary). **(c)** Target balance (≈53% up).

Figure 1: Exploratory analysis. Trending levels vs. stationary returns directly motivate the drift findings below.

Modelling. We compare a **champion** (`StandardScaler` + Logistic Regression) against a **challenger** (Random Forest, 300 trees, depth 8, regularised), both trained and evaluated under the identical chronological split with MLflow tracking. Results on the held-out test block (992 future rows) are in Table 4.

Table 4: Test-set performance. Champion selected on Accuracy/Recall/F1; both show near-random ROC-AUC.

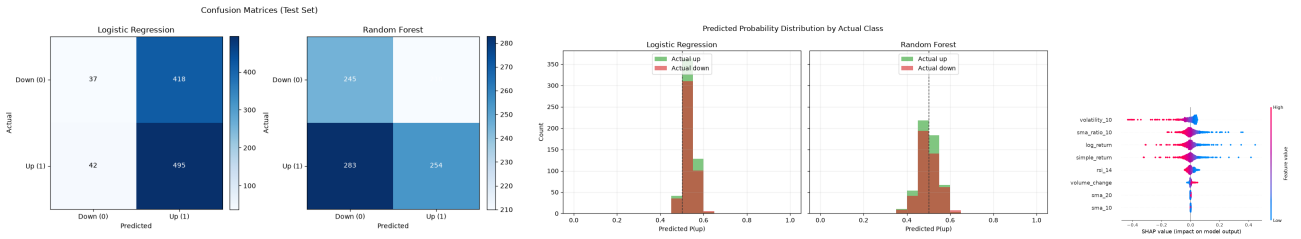
Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (champion)	**0.536**	0.542	**0.922**	**0.683**	0.499
Random Forest (challenger)	0.503	**0.547**	0.473	0.507	**0.502**



(a) Champion vs. challenger. **(b)** ROC curves (≈ diagonal). **(c)** SHAP feature importance.

Figure 2: (a) The champion leads on Accuracy/Recall/F1. (b) Both ROC curves hug the chance diagonal (AUC ≈ 0.50). (c) SHAP ranks `volatility_10` and `sma_ratio_10` highest, but all magnitudes are small.

The behavioural difference is clear in Figure 3: the champion concentrates predictions on the up-class (high recall, many false positives), while the challenger spreads errors more evenly; both predicted-probability distributions cluster tightly around 0.5, the visual signature of an appropriately uncertain model.



(a) Confusion matrices. **(b)** Predicted-probability distributions. **(c)** SHAP beeswarm.

Figure 3: (a) The champion predicts mostly "up"; (b) probabilities concentrate near 0.5; (c) per-observation SHAP values stay small and centred on the base value.

Explainability. SHAP (exact `LinearExplainer` on the champion, Figures 2c and 3c) shows the model leans most on recent risk (`volatility_10`, mean$|\text{SHAP}| = 0.035$) and distance from trend (`sma_ratio_10`, 0.032), with returns close behind, all intuitive for a short-horizon signal. Crucially, every attribution is *small*, matching the headline finding: **ROC-AUC ≈ 0.50 means neither model ranks up-days above down-days better than chance.** The champion's high recall (0.922) simply reflects it predicting "up" most days, exploiting the class prior rather than real discrimination.

Drift & conclusion. The Evidently monitor compares the oldest 70% (reference, 4,622 rows) against the newest 30% (current, 1,982 rows) with a normalised Wasserstein distance (threshold 0.1) and flags **dataset-level drift**: 4 of 8 features drift (Table 5), driven almost entirely by the non-stationary moving averages `sma_10`/`sma_20` (scores > 6), while the stationary return features do not. This is exactly the expected behaviour and pinpoints the features a production system should reformulate. Overall, the leak-free split, passing contracts, small SHAP magnitudes, and interpretable drift together confirm that the weak result is *real*, not an artifact, which is precisely what trustworthy MLOps tooling should reveal.

Table 5: Evidently drift report (Wasserstein distance, threshold 0.1). Dataset drift flagged: 4/8 features, share 0.50.

Feature	Score	Drift	Feature	Score	Drift
`sma_20`	6.154	**Yes**	`volume_change`	0.097	No
`sma_10`	6.145	**Yes**	`volatility_10`	0.071	No
`rsi_14`	0.173	**Yes**	`simple_return`	0.057	No
`sma_ratio_10`	0.105	**Yes**	`log_return`	0.057	No

5 Serving, Tracking & Reproducibility

Every training run logs its parameters, metrics, tags (e.g. `split_strategy = chronological_70_15_15`), feature list, model signature, and serialised artifact to **MLflow**. The champion is registered as `sp500_direction_model` and exposed to serving by the registry alias `candidate_champion`, so a newly promoted model is picked up without code changes. The **FastAPI** service exposes `GET /health`, `GET /model-info`, and `POST /predict` (Pydantic-validated eight-feature input $\rightarrow$ class + probability), and is packaged as the Docker image `sp500-direction-api`. Reproducibility is structural: all datasets live in the Kedro catalog, all parameters in versioned `conf/` files, random states fixed to 42, and an 80-test **pytest** suite plus the data contracts give two independent safety nets (tests catch code regressions, contracts catch data regressions).

6 From Proof of Concept to Production

This PoC runs locally: Pandas in-memory, a local MLflow server with a SQLite backend and file artifacts, a file-based feature layer, and `yfinance` for data. The technology choices are deliberately production-aligned, so the path to deployment is incremental rather than a rewrite. **Advantages of the stack:** Kedro gives modular, catalog-driven pipelines that map cleanly onto an orchestrator (Airflow/Kubeflow); MLflow already provides model lineage, versioning, and a registry the API loads by alias; Great Expectations and Evidently make data quality and drift first-class and CI-friendly; FastAPI + Docker yield a portable, horizontally scalable service; pytest plus the data contracts give two independent safety nets. Table 6 lists the main PoC limitations with proposed, effort-scoped mitigations.

Table 6: Production risks and proposed mitigations (effort as additional sprints).

Risk / limitation	Proposed mitigation	Effort
Pandas is single-node; will not scale to large/intraday data	Port feature/cleaning nodes to Spark (Kedro keeps the same node interface)	+3 wk
Local MLflow (SQLite + file store) is not multi-user or durable	Managed tracking server + cloud artifact store (e.g. S3/Postgres backend)	+1 wk
File-based feature layer, no online serving	Adopt a feature store (Feast/Hopsworks) for train/serve parity	+2 wk
`yfinance` is an unofficial, rate-limited source	Scheduled ingestion via a licensed market-data vendor + retries/back-off	+1 wk
Drift is detected but retraining is manual	CI/CD pipeline with drift-triggered, automated retraining & promotion gates	+2 wk
Weak signal (ROC-AUC ≈ 0.50)	Add alternative data (macro, sentiment, cross-asset), calibrate, walk-forward backtest	+3 wk
API has no auth / observability	Add auth, rate limiting, logging, and metrics (Prometheus/Grafana)	+1 wk

7 Packages and Versions

Developed on **Python 3.11.9**. Core dependencies (pinned versions used for the final run):

- `kedro==1.4.0`
- `kedro-datasets==9.4.0`
- `yfinance==1.4.1`
- `pandas==2.3.3`
- `numpy==2.4.6`
- `scikit-learn==1.9.0`
- `matplotlib==3.11.0`
- `mlflow==3.14.0`
- `shap==0.51.0`
- `fastapi==0.138.0`
- `uvicorn==0.49.0`
- `pydantic==2.13.4`
- `great_expectations==1.18.1`
- `evidently==0.7.21`
- `streamlit==1.58.0`
- `python-dotenv==1.2.2`
- `pytest==9.1.1`

References

[1] Yahoo Finance. S&P 500 index (^GSPC) historical data. https://finance.yahoo.com/quote/%5EGSPC/chart/